Detecting SSH Brute-Force Attacks with Wazuh in a Two-VM Lab

Rina S. Aranas | Aspiring SOC Analyst | portfolio.rinaaranas.site | github.com/thesunnymc

Objective

Build a self-contained home lab to practice the full detection lifecycle — from standing up a SIEM to generating and triaging real attack telemetry — using free, open-source tooling on commodity hardware.

Architecture

Component	Role	OS / Tooling
Ubuntu VM	Target + SIEM (all-in-one)	Ubuntu 24.04 LTS, Wazuh 4.14 (manager, indexer, dashboard)
Kali VM	Attacker	Kali Rolling, Nmap, Hydra, OpenSSH client

Both VMs run in Oracle VirtualBox on a host-only network (`192.168.56.0/24`), isolated from the internet, with a separate NAT adapter on each for package updates. This mirrors a common SOC lab pattern: an isolated segment for the target and detection stack, with controlled external access for maintenance only.

VM	Host-only IP	Resources
Ubuntu (server)	192.168.56.101	8 GB RAM / 4 vCPU / 35 GB disk
Kali (attacker)	192.168.56.102	4 GB RAM / 2 vCPU / 25 GB disk

Method

1. **Network validation** — confirmed bidirectional reachability between both VMs via ICMP and TCP (port 22) before introducing any detection tooling, to isolate network issues from application issues later.
2. **Wazuh deployment** — installed the Wazuh all-in-one stack on Ubuntu using the official installation assistant (`wazuh-install.sh -a`), which provisions the manager, indexer, and dashboard on a single host.
3. **Self-monitoring configuration** — verified that the Wazuh manager's built-in `localfile` collectors (including `journald`) were already capturing authentication events on the host itself, without requiring a separate agent package.
4. **Baseline reconnaissance** — ran an unauthenticated and version-detection Nmap scan from Kali against Ubuntu to document the pre-attack service footprint (`nmap -Pn -sV -O`).
5. **Single failed authentication** — initiated one manual SSH login with an invalid username/password from Kali to confirm the detection pipeline (journald → Wazuh manager → indexer → dashboard) was functioning end-to-end.
6. **Brute-force simulation** — used Hydra with a small custom wordlist against the same invalid username to generate a clustered burst of failed-login events, simulating a basic credential-stuffing attempt.
7. **Successful authentication** — logged in with valid credentials to capture the contrasting "normal" signature for comparison against the attack traffic.

Findings

Findings

Event	Wazuh Rule	Level	Classification
Failed login, non-existent user	5710	5	`invalid_login`
Failed login, generic	5503	5	`authentication_failed`
Successful login (valid user)	5715 / 5501	3	`authentication_success`

Key observation: Wazuh's default ruleset assigns a *higher* severity (level 5) to failed logins against non-existent usernames than to successful logins (level 3). This reflects a deliberate detection design choice — attempts against unknown accounts are a stronger indicator of malicious reconnaissance or credential-guessing than routine authentication activity, even before any brute-force volume is considered. Each alert was automatically enriched with mappings to PCI-DSS, GDPR, HIPAA, and NIST 800-53 control references, supporting compliance-oriented triage.

The Hydra run against the dashboard's alert counters showed a measurable, attributable increase in low-severity events, all correlated to the same source IP (`192.168.56.102`) within a narrow time window — the kind of clustering a SOC analyst would use to distinguish a brute-force attempt from isolated login noise.

Troubleshooting Log

Real environments rarely cooperate on the first try. Documenting failure points is as much a part of this exercise as the detection itself:

- **Stale APT package index** caused an initial `openssh-server` install to fail with `404 Not Found` against a mirror; resolved with `apt update` before retrying.
- **Disk undersized** for a comfortable Wazuh install — resized the Ubuntu virtual disk from 25 GB to 35 GB via `VBoxManage modifymedium --resize`, then extended the guest partition in-place with `growpart` and `resize2fs`, with zero data loss or reboot required.
- **Kali NAT adapter lost its default route** after reconfiguration, producing "network unreachable" on outbound pings — diagnosed via `ip route` (no `default via` entry) and resolved at the VirtualBox adapter level.
- `wazuh-agent` **package conflict** — attempted to install a standalone agent on the Wazuh server itself for local monitoring, which `dpkg` correctly rejected as conflicting with the installed `wazuh-manager` package. Root cause: the manager already ships its own local log-collection daemons: a separate agent is neither needed nor supported on the same host.

Each of these was root-caused using the tool's own diagnostic output (`dpkg` conflict messages, `ip route`, `df -h`) rather than guesswork — a habit directly transferable to SOC triage work.

Skills Demonstrated

- Linux system administration (disk/partition management, systemd/journald, package management)
- Network troubleshooting (routing, NAT vs. host-only adapters, TCP/ICMP diagnostics)
- SIEM deployment and configuration (Wazuh manager, indexer, dashboard)
- Offensive tooling for defensive validation (Nmap, Hydra)
- Log analysis and alert triage, including severity/rule-logic interpretation

- Log analysis and alert triage, including severity/rule-logic interpretation
- Clear technical documentation of both successes and failure points

Next Steps

- Add a second monitored endpoint (Windows or additional Linux host) as a true remote Wazuh agent
- Build custom detection rules for failed-login thresholds (active response / auto-block)
- Map observed alerts to MITRE ATT&CK techniques using Wazuh's built-in module
- Extend the wordlist and test against a hardened SSH configuration (key-only auth, fail2ban) to compare detection behavior